# DataBase First Com .NET 8 +EF Core + SQL Server 2012

Neste guia abordámos o processo completo de criação e conexão de uma base de dados **Stock** no SQL Server, bem como a integração do **Entity Framework Core** numa aplicação .NET. O objetivo foi mostrar, passo a passo, como criar tabelas, instalar pacotes necessários, configurar o **DbContext** e estabelecer a ligação entre a aplicação e a base de dados.


- Criação da Tabela
- Instalação dos Pacotes Necessários e Conexão do DbContext
- Conclusão

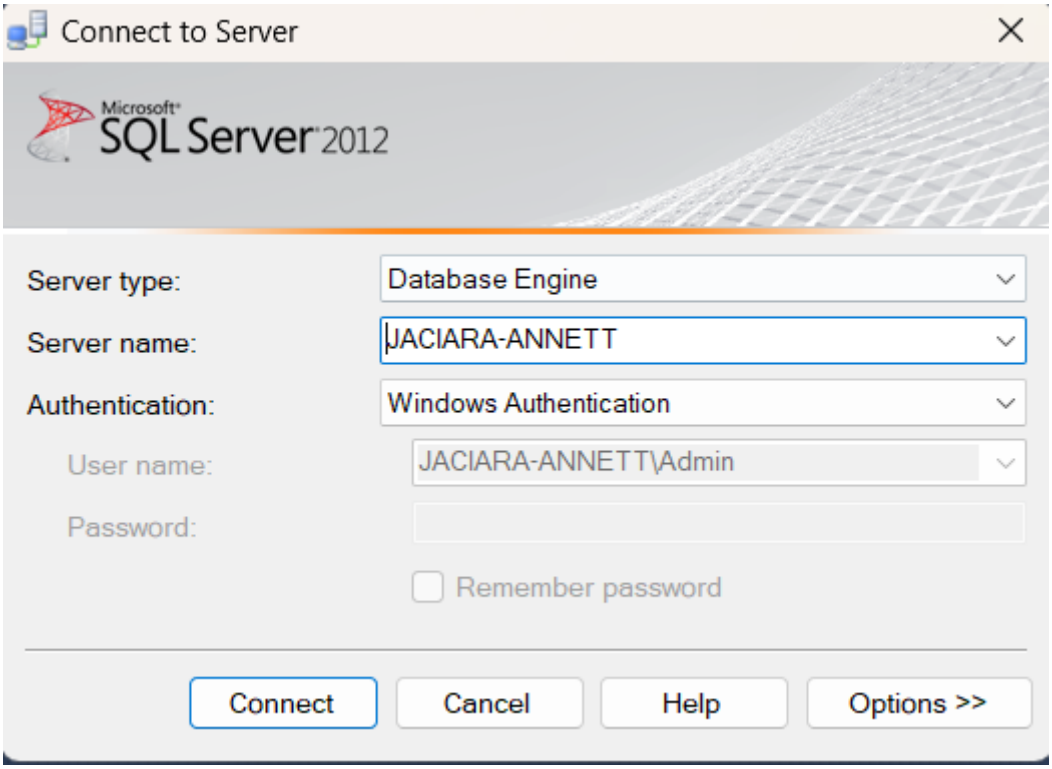
# Criação da Tabela

Neste guia iremos usar uma tabela de dados chamada **“Stock”**. Qualquer código apresentado neste exemplo pode ser **copiado e editado** conforme as suas preferências.

Caso já tenhas uma Base de Dados criada, podes avançar para a etapa de "**Instalação dos Pacotes Necessários e Conexão do DbContext**". Não te esqueças de anotar o nome do servidor onde a Base de Dados está localizada. Neste exemplo, iremos utilizar a **Windows Authentication.**

## 1. Abrir o SSMS (SQL Server Management Studio)

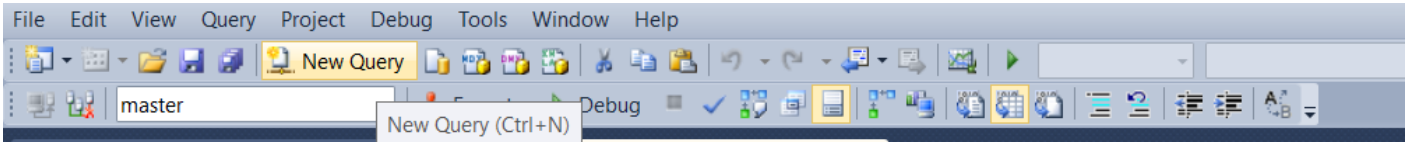
Abra o SSMS, selecione o **servidor** e o tipo de **autenticação** que irás utilizar no projeto e clique em **Conectar**.




**Nota:** Anote o **nome do servidor** e o **tipo de autenticação**, pois serão necessários posteriormente.

## 2. Criar uma nova Base de Dados

No SSMS clique em **New Query** para abrir uma nova janela de comandos.




### 3. Criar a Base de Dados e a Tabela

```
CREATE DATABASE Stock;
USE Stock;
CREATE TABLE Categoria
(
id_categoria INT IDENTITY (1,1) PRIMARY KEY,
Nome VARCHAR(50) NOT NULL,
descricao TEXT
)
```

Execute o código usando F5.

## 4. Verificar se a tabela foi criada

Para confirmar a criação da tabela, execute:

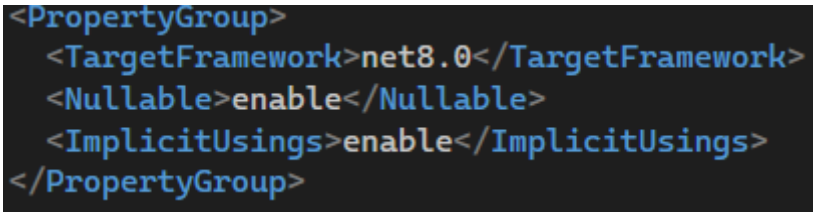
```
SELECT * FROM Categoria;
```

# Instalação dos Pacotes Necessários e Conexão do DbContext

Depois de criar a base de dados, abra ou crie o teu projeto no Microsoft Visual Studio.

## 1. Verificar a versão do .NET

No Projecto criado abra o ficheiro **StockApi.csproj** e confirme a versão do `.NET`. No meu caso, é a versão **8**.

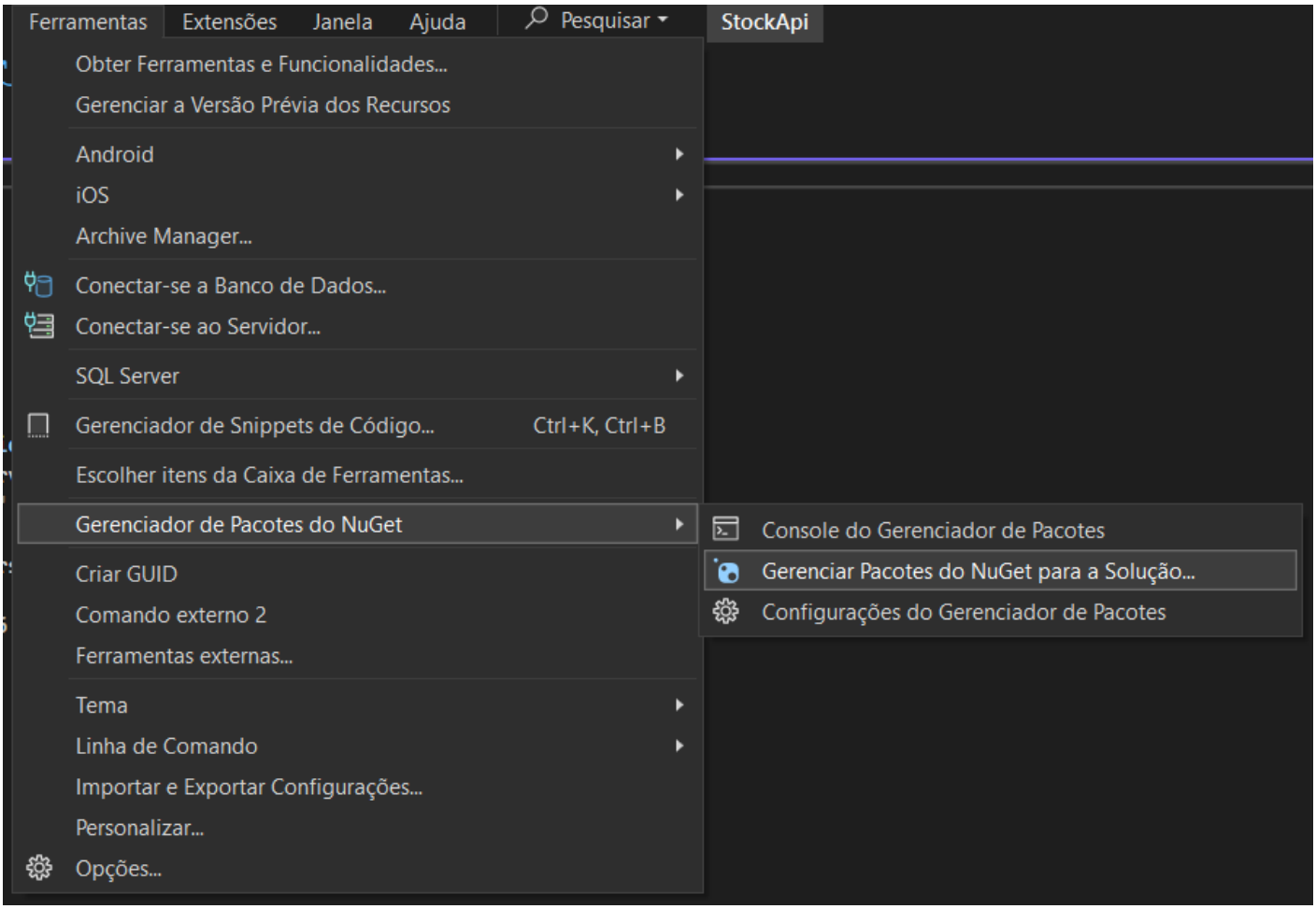



## 2. Instalar os Pacotes via NuGet

**Nota:** Esses pacotes são instalados por projetos, ou seja, tens que instalá-los sempre que iniciares um novo projeto.

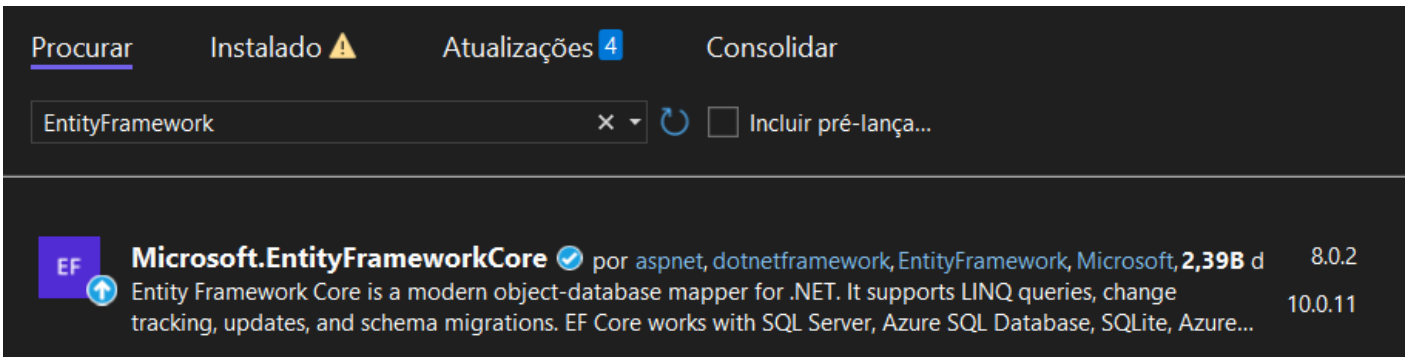
Vá em:

**Tools (Ferramentas) → NuGet Package Manager (Gerenciador de Pacotes do NuGet) → Manage NuGet Packages for Solution (Gerenciador Pacotes do NuGet para a Solução)**

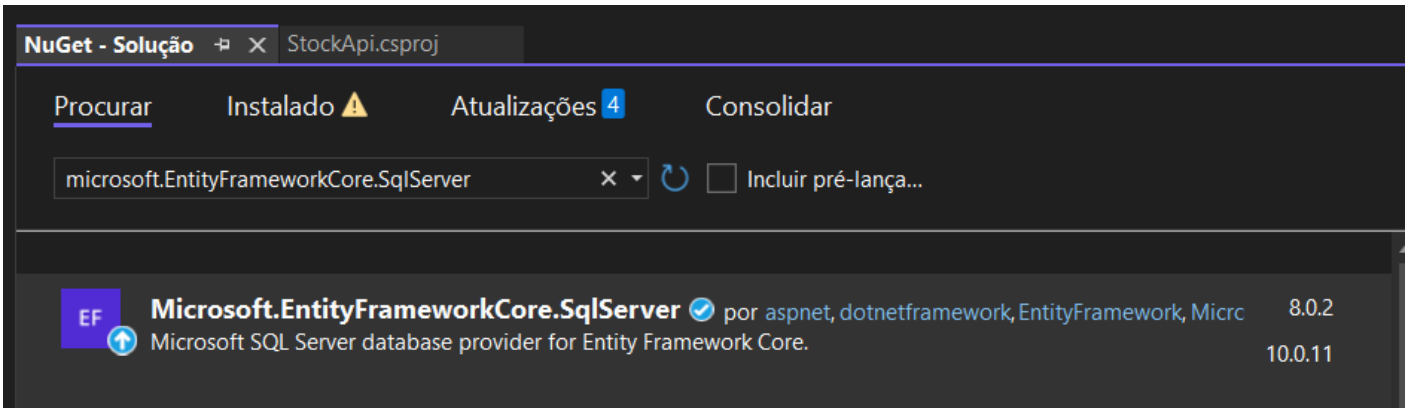



Instale os seguintes pacotes:

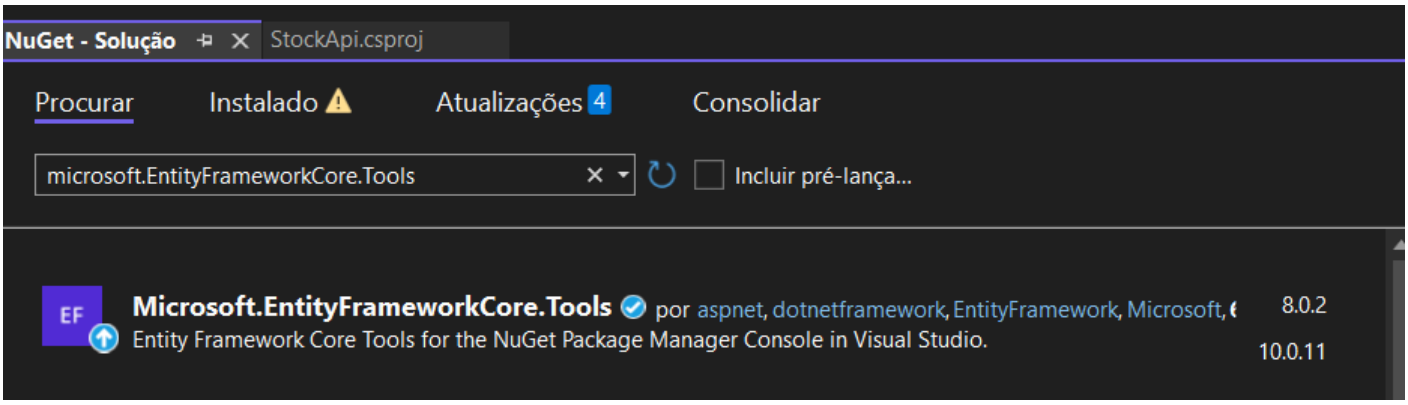
- **Microsoft.EntityFrameworkCore**



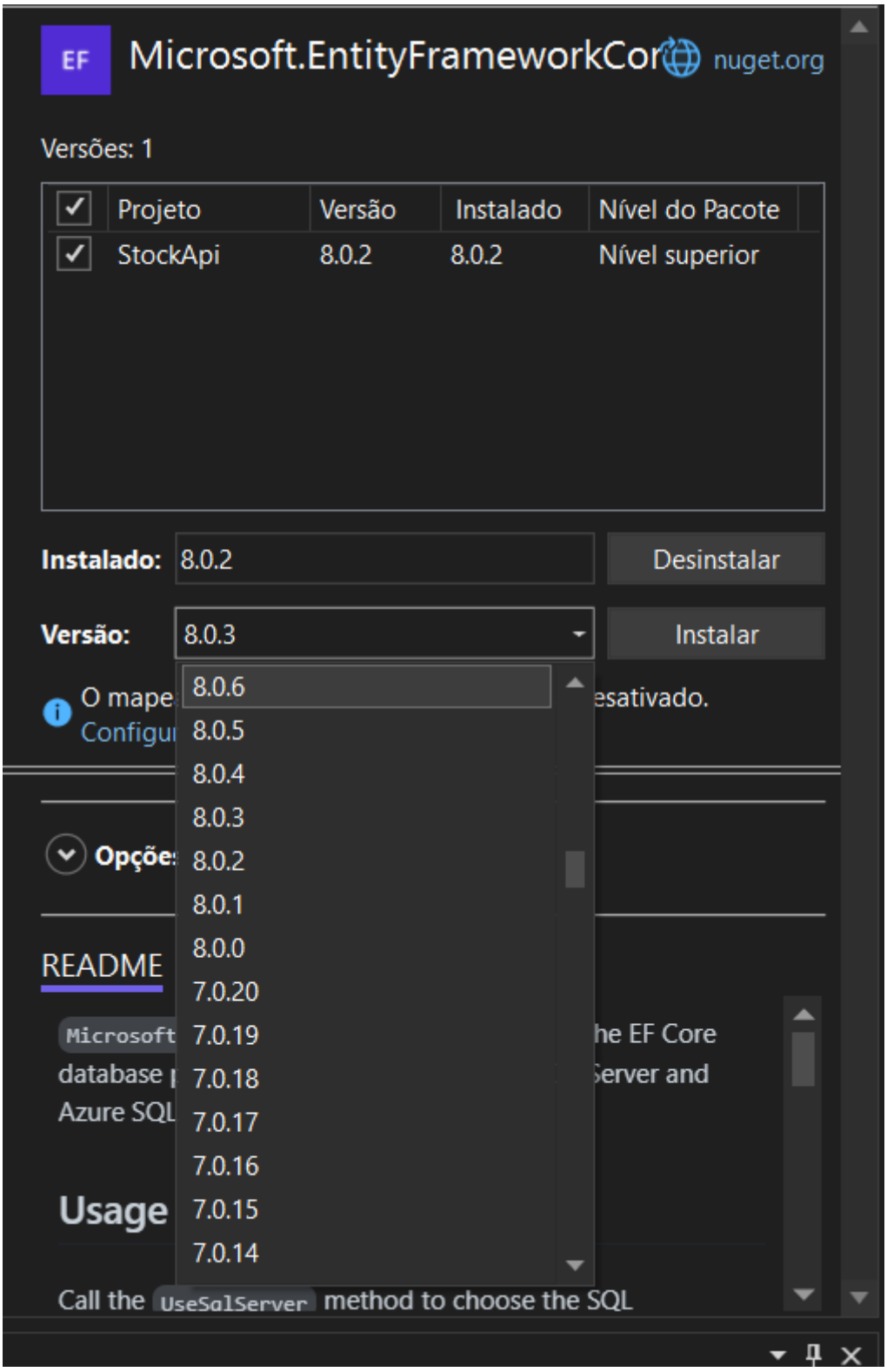

- **Microsoft.EntityFrameworkCore.SqlServer**




- **Microsoft.EntityFrameworkCore.Tools**



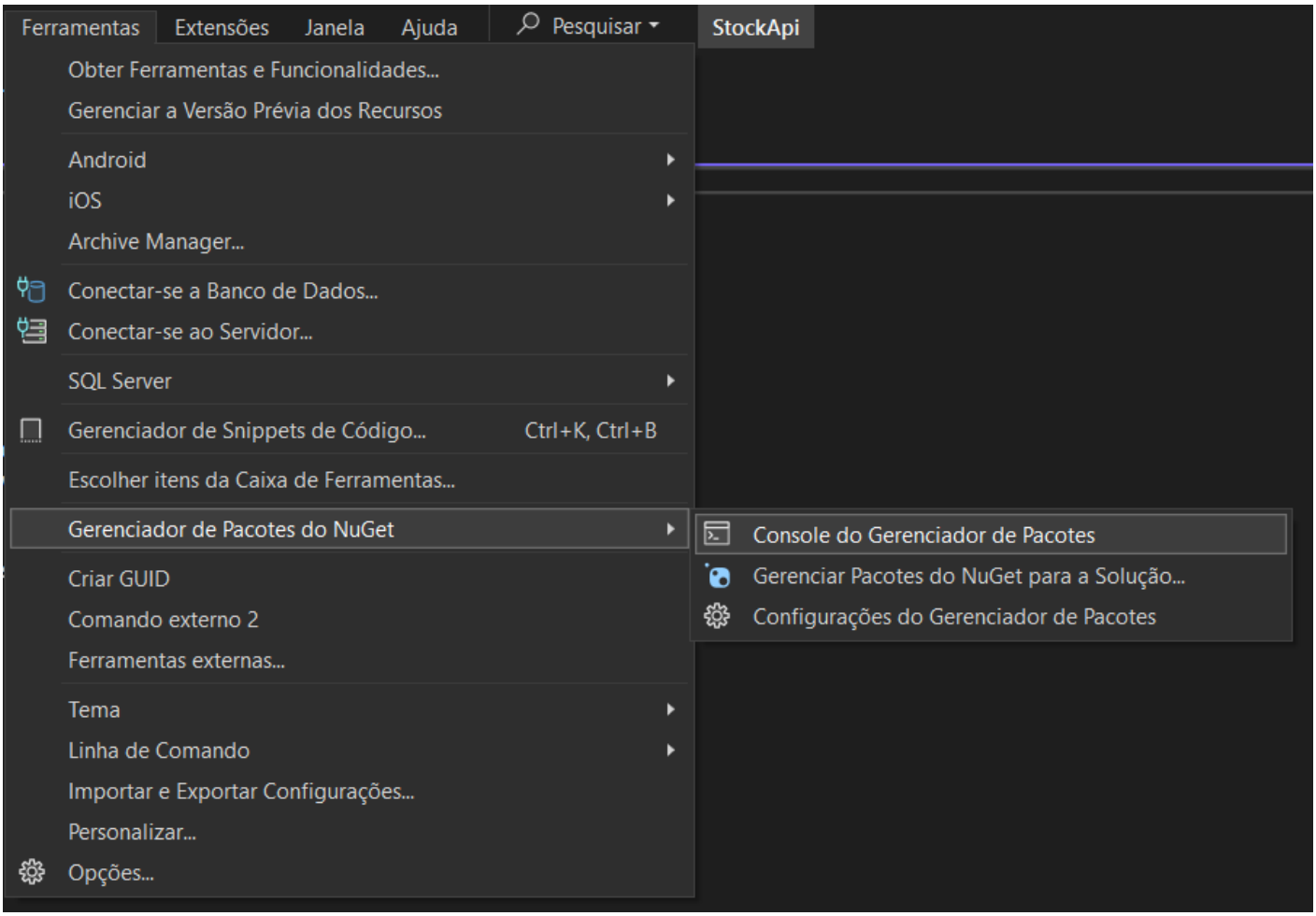

**Nota:** Mantenha todos os pacotes **EntityFrameworkCore** na mesma versão do **.NET 8.x.x** ou da versão que estiveres a usar.




## 3. Abrir o Package Manager Console

Aceda a:

**Tools (Ferramentas) → NuGet Package Manager (Gerenciador de Pacotes do NuGet) → Package Manager Console (Console do Gerenciador de Pacotes)**




Deverás ver o console aberto na parte inferior do Microsoft Visual Studio.

```
PM>
```

## 4. Executar o Scaffold

No console, execute:

```
Scaffold-DbContext "Server=Nome do
Servidor;Database=Stock;Trusted_Connection=True;TrustServerCertificate=True;"
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models/Stock -Context StockDbContext
```

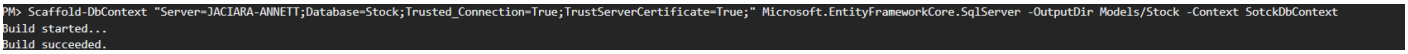
## Explicação dos parâmetros:

- **Server=Nome do Servidor;Database=Stock;Trusted_Connection=True;TrustServerCertificate=True** Indica ao Entity Framework que estamos a usar SQL Server, com autenticação Windows, e que a base de dados é **Stock**.
- **Microsoft.EntityFrameworkCore.SqlServer** Define o provedor utilizado (SQL Server).

- **-OutputDir Models/Stock** Especifica a pasta onde serão geradas as classes das tabelas e do contexto. **Atenção:** Escolha bem a pasta onde quer guardar as classes.
- **-Context StockDbContext** Define o nome da classe de contexto que será gerada.

**Nota:** O nome do servidor é o mesmo que anotaste anteriormente. A autenticação usada neste exemplo é **Windows Authentication**.

## 5. Resultado do Scaffold




Após executar o comando, será criada automaticamente a pasta:

**Models/Stock**

Com os ficheiros:

- **Stock.cs** (modelo das tabelas)
- **StockDbContext.cs** (contexto da base de dados)

# 6. Conectar o novo Database Context

## Adicionar o namespace no Program.cs

No topo do ficheiro **Program.cs**, adicione:

```
using StockApi.Models.Stock;
```

**Nota:** Para verificar o namespace a ser utilizado, abra o ficheiro **StockDbContext.cs** e copie exatamente o namespace que aparece lá.




## Registrar o DbContext no Program.cs

Após o comentário:

```
// Add services to the container.
```

Adicione:

```
builder.Services.AddDbContext<StockDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
```

## 7. Adicionar a Connection String no appsettings.json

Antes da secção `"Logging"`, coloque:

```
"ConnectionStrings": {
  "DefaultConnection": "Server=Nome do
Servidor;Database=Stock;Trusted_Connection=True;TrustServerCertificate=True"
},
```

**Atenção:** Não te esqueças de substituir o nome do **Server** e da **Base de Dados** conforme a tua configuração.

# Conclusão

Seguindo estas etapas, conseguimos:

- Criar a base de dados e a tabela **Categoria** no SQL Server.
- Instalar e configurar os pacotes do **Entity Framework Core** compatíveis com o .NET 8.
- Gerar automaticamente as classes e o contexto da base de dados com o comando **Scaffold-DbContext**.
- Registrar o **DbContext** no `Program.cs` e configurar a **Connection String** no `appsettings.json`.

Com esta configuração, a aplicação passa a ter acesso direto às tabelas e dados da base de dados **Stock**, permitindo consultas, inserções e atualizações de forma eficiente.